UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Appunti di
Laboratorio di Algorimi e Strutture dati

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso **7**

2 Argomenti del corso **7**

3 Gestione del corso (prove intercorso) **7**

4 Esame **8**

4.1 Esame semplificato e completo 8

5 Conoscenze di base **8**

5.1 Le basi 8

5.2 Compilazione 8

5.3 Linking 8

5.4 Modularità 9

5.5 Scopo degli header file 9

5.6 Header guards 9

5.7 Puntatori (`tipo* variabile`) 9

5.7.1 Puntatori a void 9

5.7.2 Riferimenti (`tipo& variabile`) 10

5.7.3 `std::move()` 10

5.7.4 Puntatore a funzione 10

5.8 Stack 10

5.9 Heap 10

5.10 Memory leak 10

5.11 Operatori `new, delete, no naked new and delete` 11

5.11.1 New . . . 11

5.11.2 Delete . . . 11

5.11.3 No naked new & delete . . . 11

5.12 Tipo string . . . 11

6 Lezione del 11-03 **11**

6.1 Struttura modulare di un programma . . . 11

6.1.1 Struttura di un progetto . . . 13

7 Lezione del 12-03 **14**

7.1 Tipi fondamentali, Puntatori e Riferimenti . . . 14

7.1.1 Dichiarazione di variabili . . . 14

7.1.2 Tipi fondamentali in c++ . . . 14

7.1.3 Tipi const . . . 15

7.1.4 Puntatori . . . 15

7.1.5 Void pointer . . . 16

7.1.6 Riferimenti . . . 16

7.1.7 Lvalue & Rvalue . . . 18

8 Lezione del 18-03 **18**

8.1 Allocazione dinamica della memoria . . . 18

8.1.1 Esempio per la locazione . . . 20

8.2 Tipi definiti dall'utente . . . 21

8.3 Le enumeration . . . 22

8.4 Libreria iostream & funzioni di `I/O` . . . 23

8.4.1 Classe istream . . . 23

8.4.2 Classi ostream 23

9 Lezione del 19-03 **24**

9.1 Libreria string 24

9.2 Generazione Pseudo-Casuale di numeri 25

9.3 Le eccezioni 26

9.3.1 Esempio 27

9.4 Puntatori a funzione 28

9.4.1 Definizione 28

9.4.2 Chiamata a funzione 28

9.4.3 Esempio con il quick-sort (VEDI REC) 28

10 Lezione del 25-03 **32**

10.1 Classi e Oggetti 32

10.1.1 Classi 32

10.1.2 Costruttori 32

10.1.3 Distruttore 32

10.1.4 Operatori (funzioni membro) 32

10.2 Sintassi della classe 33

10.3 Sintassi dei Metodi 33

10.3.1 Definizione concreta di un metodo 33

10.4 Ereditarietà di tipo virtuale 34

10.5 Esempio 34

11 Lezione del 26-03 **34**

11.1 Template 34

11.1.1 Vector.hpp . . . 35

11.1.2 Vector.cpp . . . 35

11.1.3 main . . . 35

11.2 Differenze e similitudini con i linguaggi C e Java . . . 36

12 Lezione 08-04 09-04 **36**

13 Lezione del 15-04 **36**

13.1 Gli alberi binari . . . 36

13.2 Albero binario di ricerca . . . 37

14 Lezione del 16-04 **38**

14.1 Gli iteratori . . . 38

14.1.1 Implementazione degli iteratori sugli alberi . . . 38

15 Lezione del 22-04 **38**

16 Lezione del 23-04 **39**

17 Lezione del 29-04 - Alberi bilanciati **39**

17.1 Costruzione di un ABR dato un Linear Container . . . 39

17.2 Costruttori di copia e spostamento . . . 40

17.3 Operatori di confronto . . . 40

17.4 Exists . . . 40

17.5 `Insert/Remove` . . . 40

17.6 `Max/Min` (`Get/Remove`) . . . 40

17.7 `Predecessor/Successor` (`Get/Remove`) . . . 40

18 Lezione del 30-04 **40**

18.1 Inserimento e Remove negli alberi 40

18.2 Funzione Detouch 40

18.3 Predecessor & Successor 41

19 Lezione del 06-05 **41**

19.1 Matrice 41

19.1.1 Rappresentazione Vettoriale 42

19.1.2 Rappresentazione a lista (lineare) 43

19.1.3 Rappresentazione Compress `Row/Column` (`CSR/CSC`) 43

20 Lezione del 07-05 **44**

21 Lezione del 13-05 **44**

21.1 Recap su CSR 44

1 Introduzione al corso

`Ricevimento` → Lunedì ore 16-18 (Teams).

`Email` → fabio.mogavero@unina.it

Link al download del libro di Data Structure & Algorithm Analysis → clicca per il download

2 Argomenti del corso

- Elementi di Linguaggio C++;
- Tipi di dato astratto e relative implementazioni in C++;
- Progettazione di una libreria contenitore di dati;
- Strutture dati elementari (vettori, liste, pile, code);
- Alberi binari di ricerca e iteratori sui dati;
- Matrici e grafi.

3 Gestione del corso (prove intercorso)

- Esercizio Vettori e liste (26 Marzo - 11 Aprile);
- Esercizio Pile e Code (9 Aprile - 25 Aprile);
- Esercizio Alberi Binari di Ricerca (23 Aprile - 9 Maggio);
- Esercizio Iteratori su Alberi (7 Maggio - 23 Maggio);
- Esercizio su Matrici (21 Maggio - 6 Giugno).

Per la consegna:

- Invio file .zip o tar.gz alla mail del prof con oggetto "Consegna Esercizio x (Cognome-Nome-Matricola)"
- Formato File: Cognome-Nome-Matricola(Esercizio x).zip *o* Cognome-Nome-Matricola.tar.gz
- Scadenza improrogabili → ore 23:59 dell'ultimo giorno previsto per l'esercizio

4 Esame

Esame in presenza:

- Prova al calcolatore + discussione orale

4.1 Esame semplificato e completo

Per chi consegna tutti gli elaborati si ha la possibilità di accedere all'esame semplificato (reimplementazioni, implementazione di una `vecchia/nuova` funzionalità di una libreria tra quelle implementate).

Per chi non consegna tutti gli elaborati si è obbligati ad accedere all'esame completo (implementazione da zero di una libreria).

Prerogative per entrambe le modalità è l'obbligo della consegna di tutte le librerie con data ultima precedente alla data di esame a cui si vuole accedere.

Nel caso di modalità telematica nel caso dell'esame semplificato si svolgerà solo la parte orale.

5 Conoscenze di base

Grazie a `Umberto de Angelis` e `Simone Cerrone`

5.1 Le basi

Ogni programma in C++ ha esattamente una funzione globale chiamata `main()` di tipo intero che fallisce ritorna un valore diverso da 0.

5.2 Compilazione

A differenza dei linguaggi interpretati, la traduzione avviene una volta per tutte e non a *runtime*.

2 modalità:

- Classico build.sh (lento perchè ripete ogni volta la compilazione di tutti i file, e i tempi quindi aumentano rapidamente);
- Makefile (molto comodo, in quanto ricompila solo le cose che sono variate, rendendo la seconda compilazione molto veloce rispetto alla prima).

5.3 Linking

Si occupa del collegamento tra i file oggetto e i file di libreria.

5.4 Modularità

Generalmente è impensabile creare un programma in un singolo file, per cuo viene strutturato in più file di diverse estensioni divise in file `.hpp` (header file) ed in file `.cpp` (implementation file) che vengono compilati per creare il file oggetto.

Si assume che ogni header file che viene incluso come prototipo (la firma della funzione) nelle altre librerie inizi e termini con le macro

```
#infndef TEST_HPP
#define TEST_HPP

// Corpo della macro

#endif
```

5.5 Scopo degli header file

File che aiutano il programmatore nell'utilizzo di librerie durante la programmazione.

Contiene i prototipi delle funzioni definite nel relativo file `.cpp`

5.6 Header guards

Particolari direttive (macro) che vengono usate nei file header per evitare problemi di doppia definizione in fase di linking (risolte dal pre-processore).

5.7 Puntatori (`tipo* variabile`)

Tipi di dato che rappresentano la posizione (utilizzando indirizzi di memoria) di elementi come:

- Variabili;
- Oggetti;
- Strutture dati;
- *etc* ...

Il puntatore `nullptr` non punterà a nessun oggetto.

5.7.1 Puntatori a void

Possono puntare a qualsiasi tipi di variabile, nativa o meno, ma non potranno essere dereferenziati.

Per farlo sarà necessario una conversione esplicita tramite `cast`.

5.7.2 Riferimenti (`tipo& variabile`)

Variabili che occupano la stessa memoria delle variabili a cui si riferiscono.

Questi oggetti hanno *accesso diretto* alla variabile assegnata, quindi, qualsiasi modifica effettuata tramite riferimento modificherà il valore originale della variabilie assegnata.

5.7.3 `std::move()`

Funzione della standard library che effettua una trasformazione del tipo di valore da un `RValue` ad un `LValue`.

5.7.4 Puntatore a funzione

È un tipo di puntatore che quando viene deferenziato, invoca una funzione passando 0 o più argomenti come ad una funzione normale.

I puntatori a funzione possono essere usati per semplificare il codice fornendo un modo semplice per eseguire codice di base a parametri determinati a `run-time`.

Essi, inoltre, permettono la chiamata di funzioni diverse con lo stesso prototipo conoscendone l'indirizzo

5.8 Stack

L'area di memoria `stack` (memoria fissa in base all'*OS*) è quella in cui viene allocato un pacchetto di dati non appena l'esecuzione passa dal programma chiamante a una funzione.

Questo pacchetto viene *impilato* sopra il pacchetto precedente e poi automaticamente rimosso dalla memoria appena l'esecuzione della funzione è terminata.

5.9 Heap

L'area di memoria è soggetta a regole di visibilità e tempo di vita diverse dallo stack, e precisamente:

- L'area *heap* non è allocata automaticamente, ma può essere allocata o rimossa solo su esplicita richiesta del programma;
- Il suo scope coincide con quello del puntatore che contiene il suo indirizzo;
- Il suo lifetime coincide con l'intera durata del programma a meno che non venga esplicitamente deallocata.

5.10 Memory leak

Se il puntatore va *out of scope*, l'area non è più accessibile, ma continua a occupare memoria inutilmente, ciò potrebbe verificare l'errore di memory leak.

5.11 Operatori `new, delete, no naked new and delete`

5.11.1 New

Costruisce uno o più oggetti nell'area heap e ne restituisce l'indirizzo.

In caso di memoria non disponibile (*bad-alloc*) non restituirà `nullptr`.

5.11.2 Delete

Dealloca la memoria dell'area heap puntata dall'operando.

Non restituisce alcun valore e quindi deve essere usato da solo in un'istruzione.

Questo operatore non cancella il puntatore nè altera il suo contenuto, l'unico effetto è di liberare la memoria puntata rendendola disponibile.

L'operatore delete costituisce l'unico mezzo per deallocare memoria head.

5.11.3 No naked new & delete

È consigliato effettuare allocazione di memoria all'interno delle classi

5.12 Tipo string

In C++ non esiste un vero e proprio tipo stringa (viene vista come una sequenza di caratteri contenuta in uno spazio in memoria, il *buffer*).

6 Lezione del 11-03

6.1 Struttura modulare di un programma

Primo esempio (main.cpp)

```
#include <iostream> // libreria standard c++ I/O
// dichiarazione dello spazio dei tipi
//(accesso alle funzionalità senza la specificazione "std::")
using namespace std;
int main(){
    cout << "Hello World!" << endl;
}
```

Di seguito invece il `build.sh` (script bash)

```
#!/bin/bash
g++ [-O3] -o main main.cpp
```

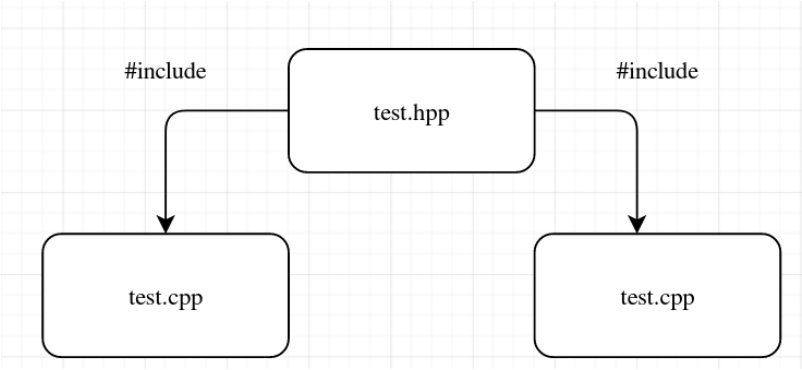
Dove:

- g++ → compilatore GNU;
- `[-O3]` → parametri di ottimizzazione opzionale;
- -o main → `output/file eseguibile`;
- main.cpp → codice sorgente

Nel momento in cui ci siano più file da compilare vanno aggiunti manualmente (l'ordine è indefferente)

6.1.1 Struttura di un progetto

Nel caso di più file si avrà un file principale più altri che eseguiranno effettivamente le operazioni:



test.hpp → file header. Si presuppone che abbia delle macro iniziali e finali che viene letto dal precompilatore

```
// main.cpp
#include <iostream>
#include "test.hpp"
using namespace std;
int main(){
    cout << "Chiamata a funzione";
    test();
}
```

```
// test.hpp
// Le macro vengono risolge dal preprocessore
#ifndef TEST_HPP
#define TEST_HPP
void test();
#endif


// test.cpp
#include <iostream>
#include "test.hpp"
void test(){
    std::cout<<"OK!"<< std::endl;
}
```

7 Lezione del 12-03

7.1 Tipi fondamentali, Puntatori e Riferimenti

7.1.1 Dichiarazione di variabili

`type varname [= defaul value]`

`type varname [numero elementi] [{` $\mathrm{val}_0, \ldots, \mathrm{val}_n$ `}]`

7.1.2 Tipi fondamentali in c++

- Bool (`true/false`);
- `[unsigned]` char;
- `[unsigned]` short;
- `[unsigned]` int;
- `[unsigned]` long int;
- `[unsigned]` long long int;
- float;
- double;
- long double

In alcuni compilatori è possibile delle abbrevazioni per tipi unsigned int e long del tipo:

- uint;
- ulong;

7.1.3 Tipi const

```
const type varname = value;
```

Ritornano utili anche nel passaggio dei parametri nelle chiamate a funzione.

7.1.4 Puntatori

```
type* varname [=default address]
```

es.

```
#include <iostream>
using namespace std;
int main(){
    char c = 'a';
    char* p = &c;
    cout << "Il valore di *p è "<< *p << endl;
    p= nullptr; //corrisponde alla macro NULL del linguaggio C
    cout << p << endl;
}
```

```
Il valore di *p è a
```

```
#include <iostream>
using namespace std;
int main(){
    const char c = 'a';
    char* p = &c; // ERRORE
    const char* p = &c; // CORRETTO
    cout << *p << endl;
}
```

7.1.5 Void pointer

```
void* varname [= ...]
```

Un void pointer è un puntatore che non è associato a nessun tipo specifico. Puo mantenere un indirizzo di ogni tipo e può essere typecasted a ogni tipo.

Indefinire un puntatore in cui non è noto a priori il tipo di dato a cui si vuole puntare.

```
#include <iostream>
using namespace std;
int main(){
    char c = 'a';
    void* p = &c;
    //cout << *p << endl; // ERRORE
    cout << *((char*)p) << endl; // CORRETTO
    cout << *(static_cast<char*>(p)) << endl; // MODO MIGLIORE
}
```

7.1.6 Riferimenti

```
type & varname = ...
```

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint & e = d;
    e++;
    cout << "Il valore di d è " << d;
}
// Stamperà il valore 8
```

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint e = d;
    e++;
    cout << "Il valore di d è " << d;
}
// Stamperà il valore 7
```

Con i puntatori invece:

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint* e = &d;
    (*e)++;
    cout << "Il valore di d è " << d;
}
```

```
Il valore di d è 8
```

```
int & i = 1; // ERRORE
const int & i = 1; // CORRETTO
```

7.1.7 Lvalue & Rvalue

Differenze

In C++ un *Lvalue* è qualcosa che punta a una specifica locazione di memoria. Un *Rvalue* invece è qualcosa che non punta da nessuna parte.

In generale gli Rvalues sono temporanei e di vita breve, mentre gli Lvalues hanno una vita lunga tanto quanto quella della variabile.

```
#include <iostream>
using namespace std;
string f(){return string ("Ciao");};
int main(){
    string var = f();
    string stringa1 = move(f());
    string && stringa2 = f(); // Riferimento Rvalue
    string stringa3 = move(stringa2); // Spostiamo il valore di stringa2 in stringa3 senza copiarla
    cout << var<< endl;
    cout << stringa1<< endl;
    cout << stringa2<< endl;
    cout << stringa3;
}
```

Ciao
Ciao

Ciao

8 Lezione del 18-03

8.1 Allocazione dinamica della memoria

Dal linguaggio C in C++:

- *malloc()* → *new*;
- *free()* → *delete*;
- *realloc* invece non ha diretta corrispondenza in C++
- **new** solleva un'eccezione;

- **new** non restituisce *nullptr* quando fallisce ("bad-alloc");
- L'uso di `new/delete` non va combinato con chiamate a funzioni `malloc/realloc/free`;
- No naked "new" "delete".
- Non considerano gli smart-pointer;

```
uint num = espressione;
type* var = new type[num]; //dimensione array
delete[] var;
// Allocazione di tipo unico
// Tra le parentesi ul potenziale del valore di default
ObjectType* var = new ObjectType();
...
delete var;
```

8.1.1 Esempio per la locazione

```
#include <iostream>
using namespace std;
int main(){
try {
    // ulong* ulptr = new ulong;  Non inizializzato
    ulong* ulptr = new ulong(5); // Inizializzato
    cout << (*ulptr)++ << endl;
    cout << *ulptr << endl;
    delete ulptr;
    //delete ulptr;
}catch (bad_alloc exc){
cout << "Quite rare exception !" <<endl;
    }
try{
const ulong arraysize = 100000000000000; // Ottengo il cout della catch
ulong* ulptr = new ulong[arraysize];

cout << ulptr[0]++ << endl;
cout << ulptr[1] << endl;
cout << ulptr[2]-- << endl;
cout << ulptr[0] << endl;
cout << ulptr[1] << endl;
cout << ulptr[2] << endl;
delete[] ulptr;

}catch(bad_alloc exc){
cout << "Quite rare exception !" <<endl;
    }
}
```

Nel caso in cui si chiami una delete due volte di seguito si andrà in contro a `free(): double free detected in tcache 2`. Per evitar ciò un workaround è quello di settare a la variabile a *nullptr*

8.2 Tipi definiti dall'utente

Le struct sono classi con accesso pubblico agli attributi e alle funzioni membro; nessuna altra restrizione.

struct strudente{ campi };

Un esempio di struct:

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

struct Studente{
    string Nome;
    string Cognome;
    string Matricola;
    uint id;
};
int main(){
}
```

Member access → StructNomeVal.Campo.

Accesso via pointer → poitnerVarName -> campo (equivalente ad (*PointerVarNome).Campo)

```
#include <iostream>
using namespace std;
struct Studente{
    uint Id;
    string Matricola = "N86000";
    string Nome = "Alan";
    string Cognome = "Turing";
    // Creo un costruttore con 0 o tutti gli argomenti
    Studente() = default;
    Studente(ulong id, string matr, string nome, string cognome){
Id = id;
Matricola = matr;
Nome = nome;
Cognome = cognome;
```

```
    }
};
int main(){
    Studente stu;
    cout << "Nome: " << stu.Nome <<endl;

    // Crea un costruttore che rimpiazza i valori definiti
    Studente stud1{2, "N860001", "Gennaro", "Esposito"};
    cout << "Nome: " << stud1.Nome << endl;
    // Necessita del costruttore sopra definito
    Studente stud2(3, "N860002", "Alonzo", "Church");
    cout << "Nome: " << stud2.Nome << endl;
}
```

8.3 Le enumeration

```
enum class Nome{elemento, ..., elementoN}; // In C++
enum Nome {...};
enum class Color{white, gray, black};
enum class Color3{white, gray, black};
enum class Color4{white, gray, black};
Colore colore = Color::white;
Color4 colore4 = Color4::white; // Corretto
Color4 colore4 = Color::white; // ERRATO!
```

In C non è possibile definire due enum con campi uguali (gli elementi sono visibili a livello globale). Gli enum possono essere confrontati e assegnati. Le variabili di tipo enumerativo possono ovviamente avere un valore iniziale di default

```
#include <iostream>
using namespace std;
enum class Colore {Bianco, Grigio, Rosso}; // C++
enum class Colore1 {Grigio, Rosso}; // C++
enum Colore2{Rosso, Giallo};
enum Colore3{Blu, Verde};
int main(){
    Colore color = Colore::Grigio;
    cout << (color < Colore::Bianco) << endl;
    cout << (color == Colore::Grigio) << endl;
}
```

Il confronto tra due enumeration diverse non è possibile (non è definito)

8.4 Libreria iostream & funzioni di `I/O`

Operatori principali » (get-from) e « (put-to). Standard stream:

- cout, cerr, clog;
- cin.

8.4.1 Classe istream

Analisi dello stato:

- *good()*;
- *eaf()*;
- *fail()*;
- *bad()*;
- *get(c)* → (char);
- *getline(p,n)* → (char + numero di caratteri da leggere);

8.4.2 Classi ostream

- *put(c)*;
- *write(p,n)* → (char + numero di caratteri da scrivere);

```
#include <iostream>
using namespace std;
struct Studente{
    uint Id;
    string Matricola = "N86000";
    string Nome = "Alan";
    string Cognome = "Turing";
    friend ostream& operator<<(ostream& outstr, const Studente& stu);
    friend istream& operator>>(istream& instr,Studente& stu);
    private:
       ulong SecureNum = 0x232523;
};

// Per la stampa dei dati
```

```
ostream& operator<<(ostream& outstr, const Studente& stu){
    outstr << "Id: " << stu.Id << " Matricola: "
   << stu.Matricola << " Nome: "
   << stu.Nome << " Cognome:" << stu.Cognome;
}
// Per l'inserimento dei dati
istream& operator>>(istream& instr,Studente& stu){
    instr >> stu.Id >>  stu.Matricola >> stu.Nome >> stu.Cognome;
    instr >> stu.SecureNum;
}
int main(){
Studente stud;
    cout << stud << endl;
}
```

9 Lezione del 19-03

9.1 Libreria string

- string (`#include <string>`) diverso da string literals C e C++ (char*);
- `string nome = "Alan";`
  - uguale a `string nome("Alan");`
  - uguale a `string nome = {"Alan"}`
- Gli oggetti di tipo string sono confrontabili lessico-graficamente;
- Operator di `input/output` → « , » (via overloading);
- Presenta i metodi di
  - size();
  - empty();
  - [i] (prendere l'iesimo elemento della stringa);
  - front();
  - back();
- La concatenazione avviene con l'operatore `+`;
  - `string var = var + var2;`
  - `string var += var1;`
- var.substr(i,n) → sottostringa da posizione i a posizione i + n - 1 ;

es

```
#include <iostream>
#include <string> //opzionale nel caso si usi iostream

using namespace std;

int main(){
string Stringa1 = "Alan Turing";
string Stringa2 = "Kurt Godel";

cout << Stringa1 << " - " << Stringa2 << endl;

cout << "Lexicographic comparison (Stringa1 < Stringa2)" << (Stringa1 < Stringa2) << endl;

cout << "Lexicographic comparison (Stringa1 > Stringa2)" << (Stringa1 > Stringa2) << endl;
cout << "Lexicographic comparison (Stringa1 == Stringa2)" << (Stringa1 ==  Stringa2) << endl;
cout << "Lexicographic comparison (Stringa1 !=  Stringa2)" << (Stringa1 != Stringa2) << endl;

//cin >> Stringa2; // prende soltato tutti i caratteri fino inserimento di uno spazio / tab
//getline(cin , Stringa2); // Per prendere tutto


cout << "Size: " << Stringa1.size() << endl;
cout << "È vuota? " << Stringa1.empty() << endl;
cout << "Primo carattere: " << Stringa1.front() << endl;
cout << "Ultimo carattere: " << Stringa1.back() << endl;
cout << "Concatenazione: " << (Stringa1 + "  " + Stringa2) <<endl;

Stringa1.clear(); // Svuoto la stringa
}
```

9.2 Generazione Pseudo-Casuale di numeri

In C

```
#include <stdlib.h>
#include <time.h>
int main(){
    /* Nel caso in cui non venga settato il seed
    si avrà sempre la stessa sequenza */
    srand(time(NULL));
    for(uint i = 0; i < 15; i++){
```

```
printf("%d\n", rand());
    }
}
```

In C++

```
#include <iostream>
#include <random>
using namespace std;
int main(){
default_random_engine gen (random_device{}());
uniform_int_distribution<uint> dist(7,35);

    for(uint i = 0; i < 15; i++){
cout << dist(gen) << endl;
    }
}
```

9.3 Le eccezioni

Struttura di gestione elementare

```
try{
throw SomeException();
    }catch(SomeException exe){
    //gestione dell'eccezione
    }catch(...){
    throw} // Risolleva l'eccezione
```

`type f(parameters) noexcept` → la funzione non solleva alcune eccezione. Eccezioni utili della standard library:

- `logic_error`;
  - `lenght_error`;
  - `out_of_range`;
- `runtime_error`;
  - `overflow_error`;
  - `underflow_error`;
- `bad_alloc`;

In C++ non è previsto il `finally` → dipende dalla gestione della memoria

9.3.1 Esempio

```
#include <iostream>
using namespace std;

int main() {

  cout << "Hello everyone... I am rising some exception!" << endl;

  try {

     throw logic_error("Some logic error.");
     throw length_error("A lenght error occurred!");
     throw out_of_range("An out-of-range access to some structure occurred!");

     throw runtime_error("Some runtime error.");
     throw overflow_error("An overflow occurred!");
     throw underflow_error("An underflow occurred!");

     throw bad_alloc(); // Just an example, do not throw thsi exception.

     throw 25;

  } catch (length_error& exc) {

    cout << "Length error: " << exc.what() << endl;

    // throw;

  } catch (logic_error& exc) {

    cout << "Logic error: " << exc.what() << endl;

  } catch (overflow_error& exc) {

    cout << "Overflow error: " << exc.what() << endl;

  } catch (runtime_error& exc) {

    cout << "Runtime error: " << exc.what() << endl;
```

```
  } catch (bad_alloc& exc) {

    cout << "Bad allocation: " << exc.what() << endl;

  } catch (exception& exc) {

    cout << "All unmanaged standard exceptions reach this point!" << exc.what() << endl;

  }
  catch (...) {

    cout << "All unmanaged non-standard exceptions reach this point!" << endl;
  }
}
```

9.4 Puntatori a funzione

Possibili usi:

- Passaggio di funzione come parametro di altre funzioni;
- Funzioni di callback;

9.4.1 Definizione

In C:

```
typedef type (*FuncName)(lista parametri);
```

In C++:

```
typedef std::function <type(lista parametri)> FuncName;
```

Richiede `#include <functional>`

9.4.2 Chiamata a funzione

`*FuncName(...)` o `FuncName(...)`

9.4.3 Esempio con il quick-sort (VEDI REC)

```
#include <iostream>
```

```
#include <functional>

using namespace std;

void quicksort(int*, uint);
void quicksort(int*, uint, uint);
uint partition(int*, uint, uint);
enum class ComparisonType { LessThan, Equal, GreaterThen};


// // Function pointer a la C
// typedef ComparisonType (*CompareFunction) (int, int);

// Function pointer a la C++
typedef function<ComparisonType(int, int)> CompareFunction;

void quicksort(int*, uint, CompareFunction);
void quicksort(int*, uint, uint, CompareFunction);
uint partition(int*, uint, uint, CompareFunction);

ComparisonType OrdA(int a, int b) {
  if (a < b) {
    return ComparisonType::LessThan;
  } else if (a > b) {
    return ComparisonType::GreaterThen;
  }
  return ComparisonType::Equal;
}

ComparisonType OrdB(int a, int b) {
  if (a < b) {
    return ComparisonType::GreaterThen;
  } else if (a > b) {
    return ComparisonType::LessThan;
  }
  return ComparisonType::Equal;
}

ComparisonType OrdC(int a, int b) {
  if (a % 2 != b % 2) {
```

```
    return ((a % 2 == 0) ? ComparisonType::LessThan : ComparisonType::GreaterThen);
  } else if (a < b) {
    return ComparisonType::LessThan;
  } else if (a > b) {
    return ComparisonType::GreaterThen;
  }
  return ComparisonType::Equal;
}

int main() {

  int A[11] = {5, 7, 6, 8, 4, 9, 3, 10, 2, 0, 1};

  quicksort(A, 11);

  quicksort(A, 11, OrdA);
  quicksort(A, 11, OrdB);
  quicksort(A, 11, OrdC);

  for (uint i = 0; i < 11; i++) { cout << A[i] << ' '; }; cout << endl;

  return 0;
}

void quicksort(int* A, uint size) {
  quicksort(A, 0, size - 1);
}

void quicksort(int* A, uint p, uint r) {
  if (p < r) {
    uint q = partition(A, p, r);
    quicksort(A, p, q);
    quicksort(A, q + 1, r);
  }
}

uint partition(int* A, uint p, uint r) {

  int x = A[p];
  int i = p - 1;
```

```
  int j = r + 1;

  do {

    do { j--; } while ( x < A[j] );

    do { i++; } while ( A[i] < x );

    if (i < j) { swap(A[i], A[j]); } // "swap" is standard-library function

  } while (i < j);

  return j;

}

 void quicksort(int* A, uint size, CompareFunction cmp) {
   quicksort(A, 0, size - 1, cmp);
 }

 void quicksort(int* A, uint p, uint r, CompareFunction cmp) {
   if (p < r) {
     uint q = partition(A, p, r, cmp);
     quicksort(A, p, q, cmp);
     quicksort(A, q + 1, r, cmp);
   }
 }

 uint partition(int* A, uint p, uint r, CompareFunction cmp) {

   int x = A[p];
   int i = p - 1;
   int j = r + 1;

   do {

     do { j--; } while ( cmp(x, A[j]) == ComparisonType::LessThan );

     do { i++; } while ( cmp(x, A[i]) == ComparisonType::GreaterThen );
```

```
    if (i < j) { swap(A[i], A[j]); } // "swap" is standard-library function

  } while (i < j);

  return j;

}
```

10 Lezione del 25-03

10.1 Classi e Oggetti

10.1.1 Classi

```
class ClassName{
private:
    // valore di default definito non esplicitamente
    // attributi e funzioni membro privati
    // accessibili solo all'interno della Classe
protected:
    // Attributi e metodi visibili solo ad altre classi nella gerarchia

public:
    // Attributi e metodi visibili a tutti
}
```

10.1.2 Costruttori

```
ClassName(); // Costruttore di default
ClassName(parametri); // Costruttore specifico
ClassName( const ClassName&); // Copy constructor
ClassName(ClassName&&) noexcept; // Move constructor
```

10.1.3 Distruttore

```
~ClassName(); // Gestisce la memoria dinamica
```

10.1.4 Operatori (funzioni membro)

Es

```
bool operator == (const ClassName&) const noexcept; // Comparazione
ClassName &operator = (const ClassName&); // Copy assignment
ClassName &operator = (ClassName&&) noexcept; // Move Assignement
```

10.2 Sintassi della classe

```
class ClassName : [virtual] [private/protected/public] BaseClassName;{
    ...
}
```

Il valore di default è il *private ClassName* → classe derivata da *BaseClassName*

In C++ (a differenza di Java dove viene simulato) è presente l'eredarietà multipla.

Si può ereditare in 3 modalità diverse:

- *private* → tutto ciò che è accessibile dalla classe madre è utilizzabile dalla classe;
- *protected*;
- *public*;

10.3 Sintassi dei Metodi

```
[virtual] type NameFunc (parameterList) [const] [noexcept] [override] = [assignment]
```

Pseudo assignment → = *0* ; = *default* ; = *delete*. Dove:

- *0* → il *pure virtual*;
- *default* → il default implementation (costruttore e distruttore) di `default/copy/move`;
- *delete* → delete di un metodo esistente.

10.3.1 Definizione concreta di un metodo

```
type ClassName:: FuncName(parametres)[specifiers]{
    ...
    code
    ...
}
```

10.4 Ereditarietà di tipo virtuale

Un problema ricorrente con l'ereditarietà multipla è la definizione di classi derivate a partire da classi base che condividono un antenato comune (il cosidetto `diamond problem`)

La ridondanza può essere eliminata ricorrendo all'eredità virtuale.

L'effetto della parola chiave virtual in una clausola di derivazione è quello di forzare il compilatore a includere la base virtuale una sola volta nella definizione degli oggetti derivati, anche se essa appare più volte nella catena di derivazione. In questo modo si ottimizza l'uso delle risorse, e si risolvono a monte eventuali conflitti di nomi.

10.5 Esempio

```
#include <iostream>
using namespace std;
class A{
    protected:
uint size = 0;
char* str = nullptr;
    public:
A() = default;
A(uint num){
    cout << "Nuovo oggetto creato" << endl;
    size = num;
    str = new char[num]();
    }
~A(){
    delete[] str;
    cout << "Oggetto eliminato" << endl;
}
};
int main(){

}
```

11 Lezione del 26-03

11.1 Template

Corrisponde al generics di Java.

Permette di definire classi parametriche su tipi di dato specificabili a livello di compilazione

Di solito si suddivide in header e sorgente (hpp e cpp). Nei template non è possibile compilare il file cpp in isolamento

11.1.1 Vector.hpp

```
template <typename Data>
class Vector{
    private:
uint size = 0;
Data* Element = nullptr;
    public:
Vector()= default;
Vector(uint n); // rappresentata di seguito
~Vector(); // rappresentata di seguito
...
Data& operator[](ulong)
}
#include "Vector.cpp"
```

No *naked new* and *delete* in Vector

11.1.2 Vector.cpp

```
template <typename Data>
Vector<Data>::Vector(uint n){
    Element = new Data[n];
    size = n;
}
template <typename Data>
Vector<Data>::~Vector(){
    delete[] Elements;
}
```

11.1.3 main

```
#include "Vector.hpp"
int main(){
Vector<int> vec(5);
vec[0] = 3;
cout << vec[0];
return 0;
```

```
}
```

11.2 Differenze e similitudini con i linguaggi C e Java

C++ è (quasi) un sovrainsieme del linguaggio C.

C++ è C con le classi (e overloading, riferimenti, template, eccezioni, etc.).

Poche differenze a livello semantico (costrutti validi in entrambi i linguaggi).

Java è un linguaggio agli oggetti che si è inspirato al C++. Esistono tuttavia svariate differenze a livello progettuale, tra cui:

C++	Java
No garbage collector	Garbage collector
Native executable	Virtual Machine
Allocation on stack & heap	Allocation (most) on the heap
Operator overloading	No operator overloading
Type-complete Meta Programming via Template	Type Parametrization via Generics
Eredità multipla	Eredità singola

12 Lezione 08-04 09-04

Descrizione del secondo esercizio + domande e chiarimenti sul primo esercizio

13 Lezione del 15-04

13.1 Gli alberi binari

- Accesso alla radice
  - nodi connessi tramite un collegamento, riferimento (a livello astratto);
  - foglie (nodi con 0 successori).

L'albero binario dovrà:

- Essere percorribile in pre-order

  - L'algoritmo esplora la radice dell'albero come primo nodo fino ad arrivare alle foglie, accedendo ai singoli nodi prima di proseguire nel cammino verso i livelli più bassi;

- Essere percorribile in post-order
  - La visita dell'albero parte dalle foglie per poi risalire alla radice, che è l'ultimo nodo ad essere esplorato, al contrario di quanto avviene nella visita pre-order dove la radice è il primo nodo ad essere visitato, per poi finire alle foglie dell'albero;

- Essere percorribile in Ampiezza
  - Si visita per livelli;

Nei primi 2 casi per la loro natura si sceglie una implementazione di tipo ricorsivo. Nel 3 caso scegliamo l'utilizzo di una coda.

Alberi completi

Un albero binario completo è un albero binario in cui ogni livello, tranne eventualmente l'ultimo, è completamente pieno, e tutti i nodi sono il più a sinistra possibile. Particolarmente vantaggioso in quanto $\mathbf{h = [\log n]}$.

Si sceglie in questo caso una rappresentazione vettoriale. In tutti gli altri casi si sceglie l'utilizzo di liste linkate.

Con una rappresentazione vettoriale la scelta più ovvia per l'implementazione si sceglie la visita in ampiezza.

Visita in Order

L'algoritmo di visita in-order è un particolare algoritmo usato per l'esplorazione in profondità dei nodi di un albero binario. In questo tipo di visita, per ogni nodo, si esplora prima il sottoalbero sinistro poi si visita il nodo corrente ed infine si passa al sottalbero destro. Più precisamente, l'algoritmo esplora i rami di ogni sottoalbero fino ad arrivare alla foglia più a sinistra dell'intera struttura, solo a questo punto si accede al nodo. Terminata la visita del nodo corrente si procede poi con l'esplorazione del sottoalbero a destra, visitando sempre i nodi a cavallo dell'esplorazione del sottoalbero sinistro e quello destro.

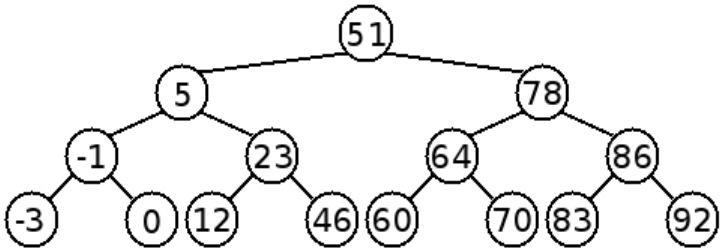
13.2 Albero binario di ricerca

Permette di effettuare in maniera efficiente operazioni come: ricerca, inserimento e cancellazione di elementi.

Un albero binario di ricerca ha le seguenti proprietà:

- Il sottoalbero sinistro di un nodo x contiene soltanto i nodi con chiavi minori della chiave del nodo x;

- Il sottoalbero destro di un nodo x contiene soltanto i nodi con chiavi maggiori della chiave del nodo x;
- Il sottoalbero destro e il sottoalbero sinistro devono essere entrambi due alberi binari di ricerca.



14 Lezione del 16-04

14.1 Gli iteratori

Un iteratore è un oggetto che viene costruito a partire da una particolare struttura dati, con il quale sarà poi possibile scorrere la struttura in un determinato ordine.

Presenta quindi:

- Un costruttore;
- Un distruttore;
- Un Access Operator(`*`, `->`);
- Test di terminazione;
- Operatore successore (`++`);
- Operatore predecessore (`--`);

14.1.1 Implementazione degli iteratori sugli alberi

Caso di visita in Ampiezza → per l'accesso con l'iteratore:

```
itr Ampiezza ::  A (*itr)
```

Con l'incremento dell'iteratore avrò il valore del nodo corrente → A → `++itr` → B → `++itr` → C e così via.
RIPRENDERE LA REC

15 Lezione del 22-04

16 Lezione del 23-04

17 Lezione del 29-04 - Alberi bilanciati

Utilizzeremo gli ABR con rappresentazione in memoria attraverso il BinaryTreeLink (possibile fare anche con il BinaryTreeVec, ma comporta costi maggiori).

Dato un albero di ricerca i metodi principali sono:

- Ricerca;
- Confronto;
- Rimozione e inserimento di nodi;
- Rimozione e lettura di
  - Exists;
  - `Insert/Remove`;
  - `Max/Min` (`Get/Remove`);
  - `Predecessor/Successor` (`Get/Remove`);

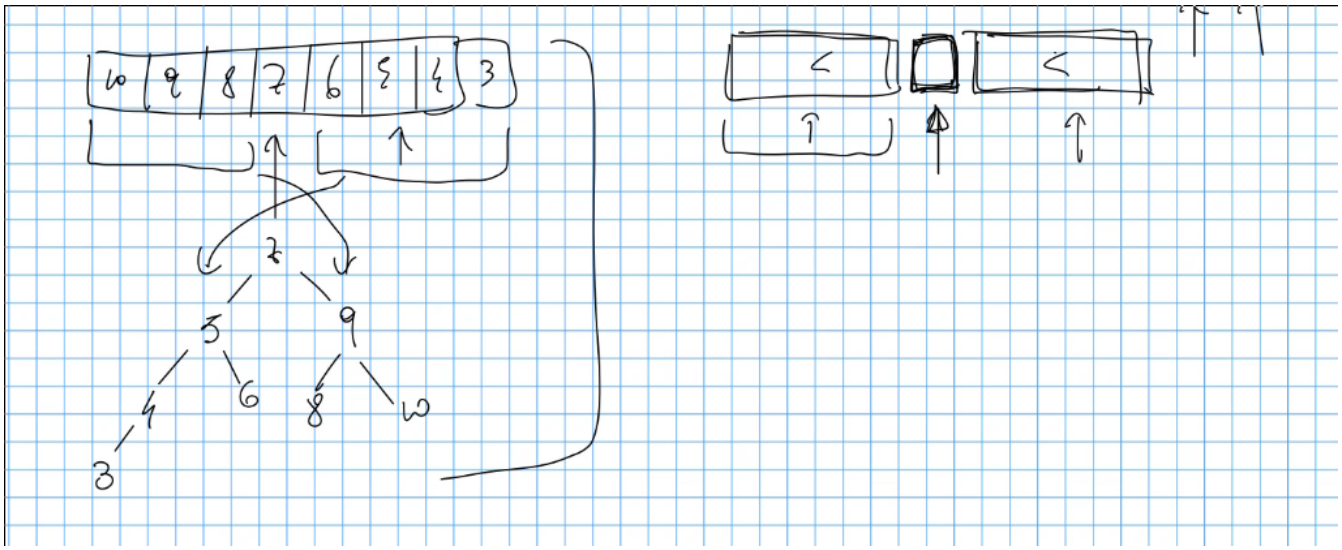
17.1 Costruzione di un ABR dato un Linear Container

Memo

$\forall$ n $\in$ T

$$\forall \text{ n} \in \text{T} \rightarrow \text{sx } \texttt{x.key < .key}$$

$$\forall \text{ n} \in \text{T} \rightarrow \text{dx } \texttt{x.key > .key}$$



17.2 Costruttori di copia e spostamento

Con questo tipo di costruttori (e anche nel caso di assegnamento di entrambi i tipi) è possibile ereditare tutti i costrutti di un ABR con BinaryTreeLink.

17.3 Operatori di confronto

Questi necessitano una reimplementazione, perchè arriviamo ad una specializzazione di alberi.

Per gestire i costi si utilizza la visita in order (che è l'unica che permette di confrontare gli alberi che strutturalmente possono essere diversi) sfruttando gli iteratori.

17.4 Exists

Basta sfruttare la ricerca binaria → Di default ricerca nell'intero albero (deve essere reimplementato). Viene in aiuto il tipo di albero in esame (è un ABR) che ha una complessità minore (logaritmica).

17.5 `Insert/Remove`

Per l'Insert si utilizza la ricerca binaria per definire la posizione in cui inserire l'elemento.

Per la Remove bisogna prima controllare che il nodo da rimuovere abbia 1 o 2 figli da rimuovere (in input avrò un dato → devo ovviamente ricercarlo).

17.6 `Max/Min (Get/Remove)`

17.7 `Predecessor/Successor (Get/Remove)`

18 Lezione del 30-04

18.1 Inserimento e Remove negli alberi

Richiedono inizialmente una fase di ricerca (`Insert(d)` e `Remove(d)`).

Serve quindi una `Accessory Function` che gestisca il caso di `assenza/presenza` del dato nella struttura → `FindPointerTo(nodo, d)` (la funzionalità della ricerca dovrà restituire un riferimento al puntatore al nodo).

18.2 Funzione Detouch

La rimozione del nodo all'interno del nodo avverà in maniera diversa in base ai casi (3):

- Nel caso di rimozione dell'albero sinistro (unico esistente), si dovrà creare una variabile locale (tmp) che punta al nodo in modo da non perderne i riferimenti nel momento in cui venga fatta la detouch. La variabile tmp poi ci permetterà di fare l'operazione di congiugimento del nodo al figlio (`SkipOnLeft`);

- Nel caso di rimozione dell'albero destri (unico esistente), si dovrà creare una variabile locale (tmp) che punta al nodo in modo da non perderne i riferimenti nel momento in cui venga fatta la detouch. La variabile tmp poi ci permetterà di fare l'operazione di congiugimento del nodo al figlio (`SkipOnRight`);
- Nel caso in cui il nodo da eliminare abbia entrambi i sottoalberi si potrà fare una `DetouchMin` (figlio destro che si sostituisce al nodo da eliminare) o una `DetouchMax` (figlio sinistro che si sostituisce al nodo da eliminare).

18.3 Predecessor & Successor

```
const Data& Predecessor(Data) // Il const per essere sicuri di non modificare il valore del dato
```

Il `Predecessor` di un dato è l'elemento che tra le chiavi più piccole ha la chiave più grande → è il massimo del sottoalbero sinistro. Se il sottoalbero sinistro è vuoto, si cerca il primo antenato che abbia come figlio destro il nodo stesso o un suo antenato.

Il `Successor` di un dato è l'elemento che tra le chiavi più grandi ha la chiave più piccola → è il minimo del sottoalbero destro. Se il sottoalbero destro è vuoto, si cerca il primo antenato che abbia come figlio sinistro il nodo stesso o un suo antenato

19 Lezione del 06-05

19.1 Matrice

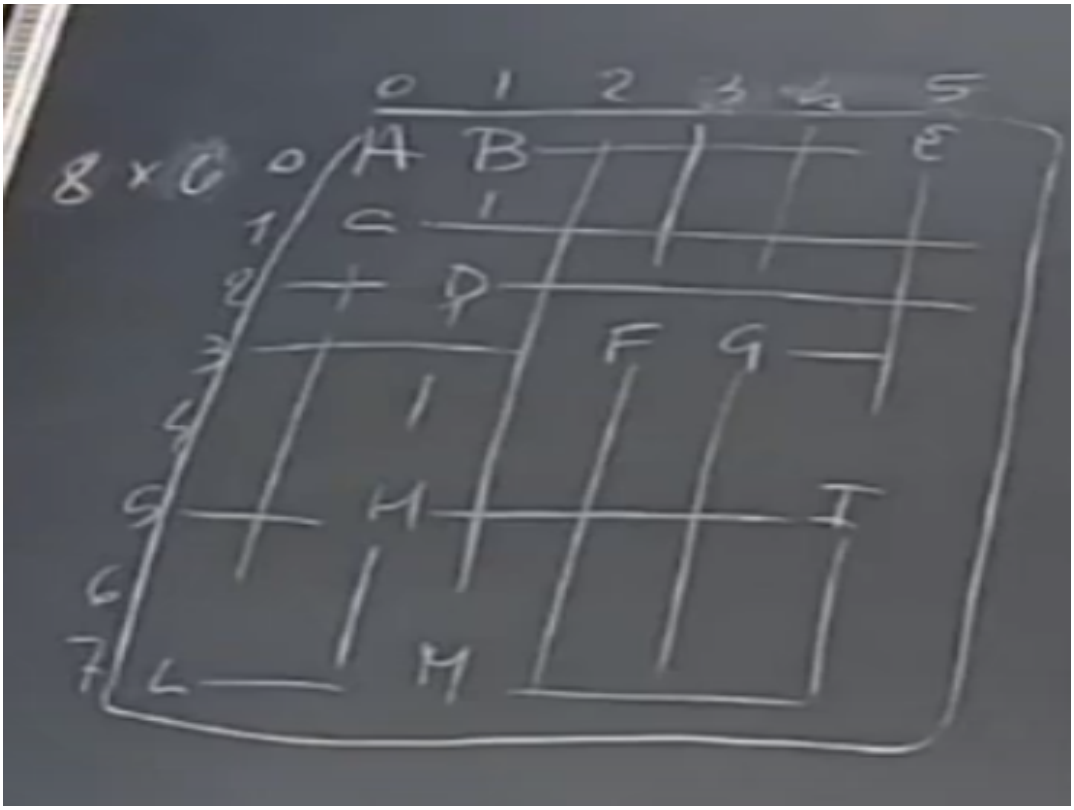
Matrice → tabella ordinata di elementi. Due tipi:

- Sparsa → una matrice con *molti* elementi pari a 0;
- Densa → una matrice con *pochi* elementi pari a 0;

Diversi tipi di rappresentazione concrete. In particolare:

- Rappresentazione *multivettoriale*;
- Rappresentazione tramite *liste ortogonali*;
- Compress `row/column` rappresentation (Yale `Format/Rapresentation`).

Supponiamo di avere una matrice `7x6` che abbia pochi dati "pieni" (gli spazi vuoti rappresentano un dato di *default* o la sua assenza):



19.1.1 Rappresentazione Vettoriale

Nel caso di rappresentazione vettoriale avremo una `size = numCol x numRighe` (array-multidimensionale) in cui verranno serializzati i dati della matrice all'interno di quest'ultimo.

Footprint in memoria → sequenza di tot dimensione di elementi. Di conseguenza ci sarà una delle 2 serializzazioni:

- Una serializzazione per riga (Row major order);
- Una serializzazione per colonna(Column major order).

Per motivi di operazioni matematiche preferisce la seconda.

L'accesso all'elemento `(i,j)` sarà:

- Nel caso di `Row major order` sarà → `i x colonna + j`;
- Nel caso di `Column major order` sarà → `j x riga + i`;

Questa rappresentazione presenta un grande spreco di spazio con matrici dense.

19.1.2 Rappresentazione a lista (lineare)

Per ovviare a questo problema si può avere una rappresentazione per triple → n° riga, n° colonna, valore del dato (o invertito con colonna, riga, valore).

In questo modo verranno occupato solo lo spazio occupato realmente (spreco di memoria minimo nel caso di matrici dense).

I `vantaggi/svantaggi` di questa rappresentazione rispetto alla precedente si riduce ai `vantaggi/svantaggi` delle strutture che permettono questo tipo di rappresentazione.

19.1.3 Rappresentazione Compress `Row/Column` (`CSR/CSC`)

Sfrutta un'idea diversa per la rappresentazione.

Sfrutta 3 vettori:

- Un vettore dei dati(A);
- Un vettore delle colonne (C);
- Un vettore delle righe (R).

The compressed sparse row (CSR) or compressed row storage (CRS) or Yale format rappresenta una matrice M per tre matrici (unidimensionali), che contengono rispettivamente valori diversi da zero, le estensioni delle righe e gli indici delle colonne.

Nel caso di una matrice:

5 0 0 0
0 8 0 0
0 0 3 0
0 6 0 0

Avremo:

- A → `[5 8 3 6]`;
- C → `[0 1 2 1]`;
- R → `[0 1 2 3 4]`.

La Compressed sparse column (CSC) è simile al CSR tranne per il fatto che i valori vengono letti prima per colonne, un indice per le righe è mantenuto in memoria per ogni valore e vengono archiviati i puntatori di colonna.

Per rendere più efficiente la ricerca all'interno di questo tipo di rappresentazione si sfrutta il fatto che il vettore

di `riga/colonna` si ordinato → si fa una ricerca binaria su questo dato (bisezione dell'array). Otteniamo una performance di ricerca di tipo logaritmica.

Un eventuale estensione di riga di questo tipo di matrice potrebbe essere addirittura a tempo medio costante (anche se nel caso di matrici un ridimensionamento oltre a essere una operazione rara, è molto costosa).

Non vale lo stesso ragionamento nel caso dell'altro indice (la colonna, nel caso di una rappresentazione di una matrice tramite compress sparse row).

Stesso ragionamento vale per una matrice con rappresentazione Compress sparse column.

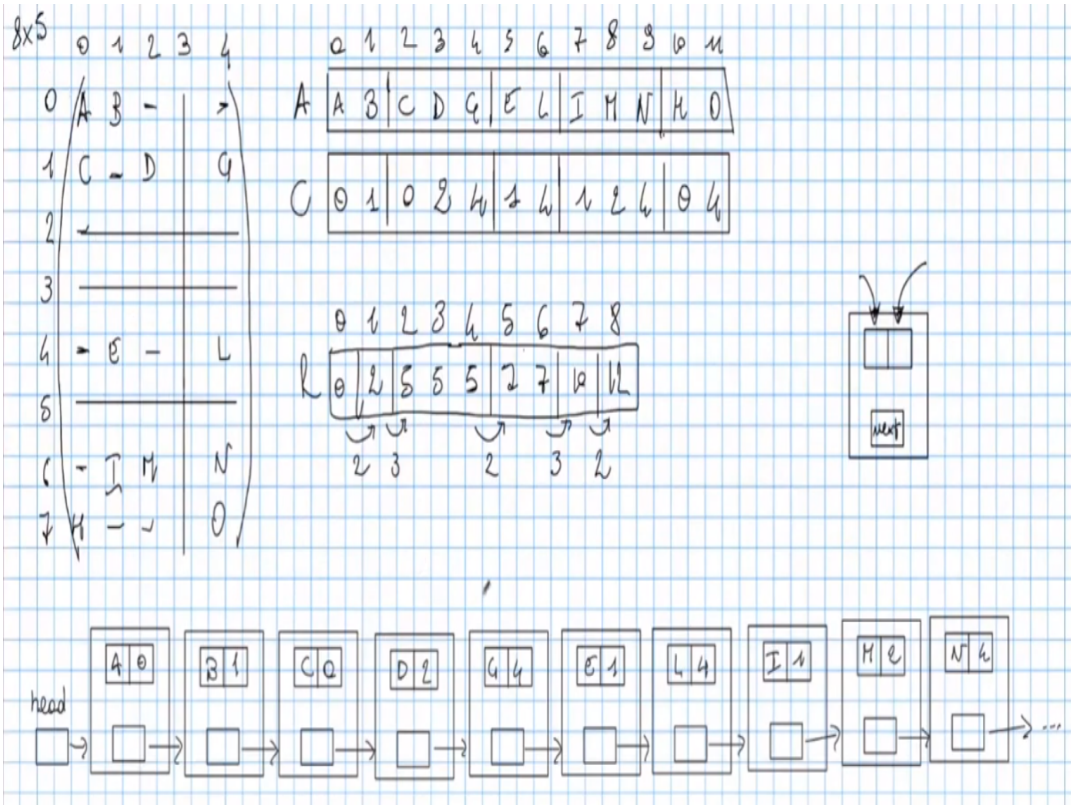
Per ovviare a questo problema si può pensare di sostituire i 2 vettori (A, C) con delle liste (con lo scopo di diminuire il costo del ridimensionamento).

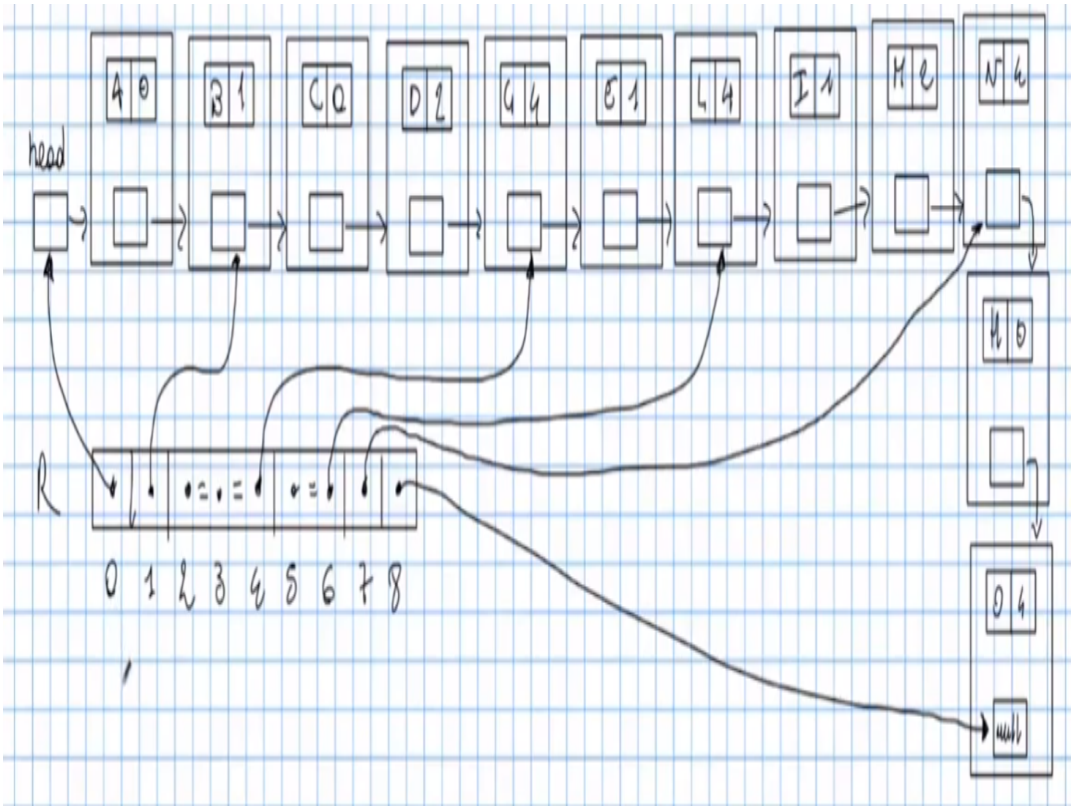
20 Lezione del 07-05

(Descrizione della libreria 4 + domande inerenti alla libreria 3/4)

21 Lezione del 13-05

21.1 Recap su CSR





Questo tipo di rappresentazione è estremamente succinta ed ha un accesso ai dati relativamente rapidi (tempo logaritmico nel numero degli elementi per riga).

Presenta il problema del ridimensionamento (nell'ampliare la struttura, nell ridimensionamento a restringere no $\rightarrow$ si potrebbe inoltre eliminare i dati senza eliminare memoria per sfruttarla poi in seguito).

Il problema quindi è quello dell'aggiunta di un elemento non precedentemente esistente $\rightarrow$ obbliga a riallocare totalmente il vettore A e C (richiede un tempo lineare).

Per ovviare a questo problema un possibile metodo è quello di allocare un maggiore spazio (richiede una gestione non banale e si va incontro anche a un certo spreco di spazio, che è quello che si cerca di evitare con questo tipo di rappresentazione).

Un altro modo è quello di sostituire i vettori A e C con delle liste (come si vede in figura). Nel vettore R ogni elemento è un puntatore al puntatore al nodo (non è diretto al nodo, perchè non permetterebbe una modifica pulita al nodo). Questa modifica permetterebbe di rendere l'inserimento di un elemento non precedentemente esistente molto meno costoso (tempo costante), ma di conseguenza la ricerca diventa per forza di cose lineare.

Per estendere il numero di righe $\rightarrow$ si fa una resize e vanno duplicati i valori terminali (per non distruggere poi successivamente la corrispondenza con le colonne).

Per diminuire il numero di righe $\rightarrow$ si fa una resize in base all'indice di riga che si vuole mantenere, di seguito si fa una delete di tutto quello che non si vuole tenere (dereferenziando ovviamente).